risp — Rapporto sui problemi del codice

Le schede dell'interprete dall'analisi di rsedit `08a853d`, verificate sulla revisione `f1ec558`

Indice

1 Che cosa contiene questo rapporto 2
1.1 Come leggere una scheda 2
1.2 Come è stato condotto l'esame 2
1.3 Riepilogo 3
2 Lettore 5
3 Valutatore e libreria di base 6
4 Documentazione e test 13
5 Appendici 14
5.1 Le schede risolte 14
5.2 Differenze da Emacs che non sono difetti 14
5.3 Parti esaminate e trovate solide 14

1 Che cosa contiene questo rapporto

Questo documento elenca i problemi trovati nel codice di risp, l'interprete Lisp di rsedit. È separato di proposito dal manuale, `architettura-interprete.typ`: il manuale descrive come il codice funziona e resta valido anche dopo una correzione; questo rapporto descrive dove il codice non fa ciò che dovrebbe. Quando un problema viene risolto, la sua scheda passa nell'appendice delle schede risolte, con il commit che lo ha risolto.

Le schede LET (il lettore) e INT (il valutatore e la libreria di base) vengono dall'analisi di rsedit della revisione `08a853d`, quando l'interprete era ancora la sua directory `core/src/lisp`. Dopo il passaggio a questo repository (`cee94f3`) sono state separate dal rapporto di rsedit, e hanno conservato i loro identificativi. Le schede DOC, sulla documentazione e i test, sono nate dalla verifica. Ogni identificativo si può citare in un commit o in una discussione; nel codice lo si cita solo in un `TODO(ID)`, e solo finché la scheda è aperta. Dove una scheda rimanda a una di rsedit lo dice nel testo, per esempio «CMD-3, nel rapporto di rsedit».

Le schede sono generate da un'unica tabella di dati all'inizio del sorgente: per aggiungerne una, o per segnarne una come risolta, basta modificare quella tabella, e il riepilogo si aggiorna da solo.

1.1 Come leggere una scheda

Ogni scheda riporta:

- **Gravità**, secondo la scala qui sotto.
- **Dove**: i file e le funzioni, o i tipi, in cui sta il difetto, con i percorsi relativi alla radice del repository.
- **Verifica**: «verificato» se il comportamento è stato riprodotto eseguendo il codice (il testo dice come, e su quale revisione), «dal codice» se la conclusione viene dalla sola lettura.
- **Cosa succede**, il meccanismo del difetto; **Come riprodurlo**, quando serve; **Effetto**, ciò che vede chi usa l'interprete; **Correzione proposta**.

Tutte le schede dell'elenco sono aperte; le risolte sono nell'appendice.

Gravità	Significato
ALTA	Il processo che ospita l'interprete termina, o si blocca senza via d'uscita, per un'espressione che dovrebbe dare un errore Lisp.
MEDIA	Un comportamento sbagliato o un rallentamento di secondi in un uso realistico, senza perdita di dati.
BASSA	Casi limite, incoerenze, codice morto, commenti e documentazione superati.

1.2 Come è stato condotto l'esame

Nell'analisi di rsedit `08a853d` l'interprete è stato letto per intero, ed esercitato con:

- un **fuzzer del lettore**, su tutte le stringhe fino a tre caratteri di un alfabeto di 28 simboli sintattici e su 300 000 stringhe casuali più lunghe;
- un **fuzzer delle forme speciali**, che ha valutato 181 322 forme speciali con sintassi malformata (liste di legami sbagliate, parametri impossibili, `yield` fuori posto) senza trovare alcun panic;
- il **fuzzer delle primitive** di rsedit, che chiamava anche le primitive di base con 807 combinazioni di argomenti anomali;
- piccoli programmi che valutano espressioni, per le riproduzioni.

Le misure di tempo e di memoria sono state prese con build di release su una macchina condivisa, quindi vanno lette come ordini di grandezza.

1.2.1 La verifica sulla revisione `f1ec558`

Ogni scheda è stata poi verificata sulla revisione `f1ec558`. Il codice che descrive è stato riletto, e la storia di git ha detto quale commit lo ha cambiato. Quasi tutte le riproduzioni sono state eseguite di nuovo, con un

test temporaneo che valuta le espressioni della scheda in un contesto che consuma carburante; le misure che non sono state ripetute, la scheda lo dice. Il campo **Dove** nomina ora file e funzioni, non righe.

Delle 29 schede dell'interprete, cinque sono risolte: LET-1, LET-2, LET-3, INT-1 e INT-5. Una lo è in parte e resta aperta con il testo aggiornato a ciò che manca: INT-13. La verifica ne ha trovate quattro nuove: INT-24, DOC-1, DOC-2 e DOC-3. Tutte le correzioni sono già nella revisione `b2d000b`, quella che rsedit usa; dopo, `039ca89` ha cambiato solo commenti e `f1ec558` ha aggiunto una variante di `LispExp` senza usarla. Le schede aperte qui sono quindi aperte anche per rsedit.

1.3 Riepilogo

> Sono aperti **28 problemi**: **1 di gravità alta**, 11 di gravità media e 16 di gravità bassa; 26 sono stati riprodotti eseguendo il codice. Altri 5 sono stati risolti dopo l'analisi, e sono nell'appendice.

Le correzioni del lettore (LET-1, LET-2, LET-3) e del carburante (INT-1, INT-5) hanno tolto quasi tutti i modi in cui un'espressione scritta male bloccava o chiudeva l'editor che ospita l'interprete. Resta lo stack: una ricorsione profonda (INT-2, l'unico problema di gravità alta) e una struttura annidata costruita da un programma (INT-4) lo esauriscono, e il processo termina. Per la memoria restano le chiusure che si tengono in vita a vicenda (INT-6) e `make-string` senza un limite (INT-13).

La tabella seguente elenca i problemi aperti, dal più grave; l'identificativo rimanda alla scheda.

ID	Problema	Gravità	Verifica
INT-2	Ricorsione profonda: stack overflow e chiusura del processo	ALTA	verificato
INT-3	`setq` su una variabile non definita dentro una funzione o un `let` crea una locale	MEDIA	verificato
INT-4	Annidamento profondo in valutazione, rilascio, `equal` e stampa: stack overflow	MEDIA	verificato
INT-6	Cicli di `Arc` tra chiusure e ambienti: memoria persa	MEDIA	verificato
INT-7	`(eq x x)` falso per lambda, atomi, fiber e primitive	MEDIA	verificato
INT-8	Una fiber non è `eq` né `equal` a sé stessa	MEDIA	verificato
INT-9	`resume` di una fiber in esecuzione la marca come finita	MEDIA	verificato
INT-10	`yield` dentro `condition-case`: catturato come errore, la fiber non si sospende	MEDIA	verificato
INT-11	Backquote: coda puntata, mappe e annidamento non elaborati	MEDIA	verificato
INT-12	`format` interpreta `\b`, `\U`, `\.` … a tempo di esecuzione	MEDIA	verificato
INT-13	`make-string` senza limite né carburante: un conteggio grande riempie la memoria	MEDIA	verificato
INT-24	`if` ignora in silenzio le forme del ramo else dopo la prima	MEDIA	verificato
LET-4	`.9`, `1e5`, `+5` letti come simboli	BASSA	verificato
LET-5	`\r`, `"` e `,` dentro i simboli; `#'f` e `?a` simboli	BASSA	verificato
LET-6	`todo!()` e stato sporco dopo `NumberParseError`	BASSA	verificato
INT-14	Interi oltre 2^{63} stampati come 9223372036854775807	BASSA	verificato
INT-15	`add-to-list`: ambiente del chiamante, terzo argomento, O(n·m)	BASSA	verificato
INT-16	`memq` / `assq` usano `equal`; `member` copia la coda	BASSA	verificato
INT-17	`string-to-number` accetta inf/NaN; `substring` oltre la fine dà `""`	BASSA	verificato
INT-18	`spawn` restituisce una `Form` vuota	BASSA	verificato
INT-19	`bootstrap_vm` lascia `verify-math`, `verify-fiber`, `fiber-step`	BASSA	verificato
INT-20	Errore `UncorrectFunctionDefinition` fuorviante	BASSA	verificato
INT-21	Atomi senza compare-and-swap	BASSA	dal codice

ID	Problema	Gravità	Verifica
INT-22	Parametri `t` / `nil` irraggiungibili	BASSA	verificato
INT-23	`unreachable!` in `function-doc`	BASSA	verificato
DOC-1	`architettura-interprete.typ` superato: lettore, carburante, cicli, percorsi	BASSA	dal codice
DOC-2	`src/tests/reader_tests.rs` senza `mod`: non compila e non gira	BASSA	verificato
DOC-3	clippy: `approx_constant` per `-3.14` in `parser_tests.rs`	BASSA	verificato

Area	Descrizione	Alta	Media	Bassa	Risolte
LET	Lettore	0	0	3	3
INT	Valutatore e libreria di base	1	11	10	2
DOC	Documentazione e test	0	0	3	0

Tabella 1: I problemi aperti per area e gravità, e quelli risolti

2 Lettore

LET-4 — `.9`, `1e5` e `+5` sono letti come simboli

Gravità BASSA
Dove `src/parser.rs` (`Parser::next_token`: lo stato `InDotStart`, con l'intervallo `'0'..'9'` che esclude il 9, e gli stati `InNumber` e `InNumberAfterDot`, senza esponente né segno `+`), `src/base/strings.rs` (`primitive_string_to_number`)
Verifica Riprodotto sulla revisione `f1ec558`: `.9`, `1e5` e `+5` sono letti come simboli, `.5` come 0,5, e `2.5e-3` dà `NumberParseError`.

Cosa succede. Dopo un punto iniziale il lessico accetta solo le cifre da 0 a 8, quindi `.5` è un numero e `.9` un simbolo. La notazione esponenziale non esiste: `1e5` e `1e300` sono simboli, e valutarli dà `UnboundVariable`; `2.5e-3` è un errore di lettura. Anche `+5` è un simbolo. `string-to-number`, che usa `f64::from_str`, accetta invece `"1e5"`, `"inf"` e `"NaN"` (INT-17).
Correzione proposta. `'0'..='9'`; esponente e segno `+` nello stato dei numeri; stesse regole per il lettore e per `string-to-number`.

LET-5 — Stranezze del lessico dei simboli

Gravità BASSA
Dove `src/parser.rs` (`Parser::next_token`: lo stato `InSymbol`, con `'\r' => {}`, e il caso generale dello stato `Default`)
Verifica Riprodotto sulla revisione `f1ec558`: `foo\rbar` si legge `foobar`, `foo"bar"` e `a,b` sono un simbolo ciascuno, `#'f` e `?a` sono simboli.

Cosa succede. Dentro un simbolo un `\r` viene saltato senza chiuderlo (`foo\rbar` diventa `foobar`); `"` e `,` ne fanno parte (`foo"bar"` è un unico simbolo, così come `a,b`). `#'f` e `?a` sono simboli e non la funzione `f` o il carattere `a` come in Emacs Lisp. Il manuale dell'interprete dice ora che non c'è un tipo carattere né `?a`; per `#'` non c'è una decisione.
Correzione proposta. Trattare `\r` come spazio, chiudere il simbolo su `"`, `'`, `` ` `` e `,`; decidere se `#'` va supportato.

LET-6 — Un `todo!()` nel lettore e uno stato sporco dopo un numero non valido

Gravità BASSA
Dove `src/parser.rs` (`Parser::next_token`: il `todo!` del ramo `Default` alla fine dell'input, e i ritorni di `NumberParseError`)
Verifica Riprodotto sulla revisione `f1ec558`: su `1.2.3 foo` la prima chiamata a `next` dà `NumberParseError("1.2.")` e la seconda `NumberParseError("1.2..")` invece di `foo`. Il `todo!` dalla lettura del codice.

Cosa succede. Il ramo `Default` del codice dopo il ciclo chiama `todo!()` se il token non è vuoto; oggi è irraggiungibile, perché ogni passaggio a `Default` svuota prima il token. Dopo un `NumberParseError` lo stato e il token non vengono ripristinati, quindi una nuova chiamata a `next()` sullo stesso `Parser` riparte da uno stato sporco. La documentazione di `Parser::next` dice ora che dopo un errore la ripresa non è garantita, e tutti i chiamanti, in risp e in rsedit, creano un `Parser` nuovo e si fermano al primo errore: il difetto resta latente.
Correzione proposta. Sostituire il `todo!` con un errore; azzerare token e stato prima di restituire ogni errore.

3 Valutatore e libreria di base

INT-2 — Nessun limite alla profondità della ricorsione: lo stack si esaurisce e il processo termina

Gravità	ALTA
Dove	`src/eval.rs` (`eval` → `eval_step` → `eval_step_permitted` → `eval_special_form_or_call_step` → `eval_function_call_step`, che valuta gli argomenti con un `eval` annidato; il ramo `"spawn"`, che crea il thread con `std::thread::spawn`)
Verifica	Riprodotto sulla revisione `f1ec558`: con la `f` qui sotto, `(condition-case nil (f 100000) (error 'caught))` chiude il processo per stack overflow invece di restituire `caught` (build di release, su un thread di test con 2 MB di stack). Le soglie dell'analisi di rsedit (in debug tra 200 e 400 livelli, in release tra 5 000 e 6 000 sul thread principale, con 8 MB di stack) non sono state misurate di nuovo.

Cosa succede. Il trampolino elimina la crescita dello stack solo per le chiamate in coda. Una chiamata in posizione di argomento discende nello stack Rust, e non esiste un contatore di profondità.

Come riprodurlo.

```
(defun f (n) (if (= n 0) 0 (+ 1 (f (- n 1)))))
(condition-case nil (f 100000) (error 'caught))   ; stack overflow: il processo termina
```

Effetto. Un errore banale in una funzione ricorsiva scritta dall'utente chiude senza un messaggio il programma che ospita l'interprete: in rsedit l'editor, senza salvare. Uno stack overflow non è un errore Lisp, quindi nessun `condition-case` lo intercetta. Emacs, con `max-lisp-eval-depth`, solleva invece un errore.

Correzione proposta. Un contatore di profondità per thread, come quello del carburante, incrementato all'ingresso di una chiamata di funzione, con un errore oltre un limite tarato sul thread con meno stack. In aggiunta, i thread di `spawn` possono essere creati con `std::thread::Builder::stack_size`; in rsedit, lo stesso vale per i thread del WorkManager (`start_thread`), che hanno anch'essi 2 MB.

INT-3 — `setq` di una variabile mai definita crea una variabile locale che sparisce

Gravità	MEDIA
Dove	`src/eval.rs` (`eval_special_form_or_call_step`, il ramo `"setq"`: `update_variable`, altrimenti `set_variable` nell'ambiente corrente), `src/environment.rs` (`Env::update_variable`, `Env::set_variable`)
Verifica	Riprodotto sulla revisione `f1ec558`: dopo `(defun g () (setq zz 5))` e `(g)`, leggere `zz` dà `unbound-variable`.

Cosa succede. `setq` cerca la variabile lungo la catena degli ambienti. Se non la trova, la crea nell'ambiente **corrente**, cioè in quello della funzione, del `let` o della fiber che sta eseguendo, e non nell'ambiente globale. Quando la funzione ritorna, la variabile sparisce. In Emacs Lisp un `setq` su una variabile non legata imposta il valore globale. Il manuale dell'interprete lo descrive ora tra le differenze da Emacs.

Come riprodurlo.

```
(defun conta () (setq contatore 1))
(conta)
contatore                ; => UnboundVariable
```

Effetto. Una configurazione o un modulo che fanno `setq` di uno stato globale dentro una funzione, senza un `defvar` precedente, perdono quello stato in silenzio. In rsedit, da `4506432`, `M-:` e `eval-file` valutano nell'ambiente globale, quindi i loro `setq` al livello più esterno restano; dentro un `defun` o un `let` la variabile sparisce ancora. `add-to-list` ha un difetto simile per conto suo (INT-15).

Correzione proposta. Se la variabile non esiste in nessun ambiente, crearla nell'ambiente radice, come fa Emacs. Se invece la semantica attuale è voluta, va documentata in `setq` e va reso disponibile un modo esplicito (`set-default` o `defvar`) da usare dentro le funzioni.

INT-4 — Annidamenti profondi esauriscono lo stack anche fuori dal lettore

Gravità	MEDIA
Dove	`src/types.rs` (`impl Drop for ConsCell`, iterativo solo lungo il `cdr`), `src/lispexp.rs` (`impl PartialEq for LispExp`, ricorsivo sul `car`, e `impl Debug`), `src/utils.rs` (`data_to_form`), `src/eval.rs` (`eval`, `eval_backquote`)
Verifica	Riprodotto sulla revisione `f1ec558`: `(let ((x nil)) (dotimes (i 300000) (setq x (list x))) 'built)` chiude il processo per stack overflow quando il `let` rilascia `x`. Il resto dalla lettura del codice.

Cosa succede. Il rilascio di una lista e il confronto con `equal` scorrono la catena dei `cdr` in modo iterativo, ma scendono nei `car` per ricorsione. La valutazione di forme annidate e la stampa sono ricorsive.

Come riprodurlo.

```
(let ((x nil))
  (dotimes (i 300000) (setq x (list x)))
  'built)                ; stack overflow nel rilascio di x
```

Effetto. Abort del processo. Il lettore ora rifiuta più di 256 livelli di annidamento (LET-3, risolta), quindi una struttura così profonda non può venire da un testo: deve costruirla un programma, ma basta un ciclo come quello sopra.

Correzione proposta. Rilascio, `equal` e stampa con una pila esplicita; per la valutazione vale la correzione di INT-2.

INT-6 — Le chiusure che si riferiscono a sé stesse non vengono mai liberate

Gravità	MEDIA
Dove	`src/types.rs` (`Lambda`, che possiede il suo ambiente con un `Arc`), `src/environment.rs` (`Env`), `src/eval.rs` (i rami `"lambda"`, `"defun"`, `"defmacro"` e `"fiber"`, che catturano l'ambiente)
Verifica	Riprodotto sulla revisione `f1ec558`: `(dotimes (i 300000) (let ((f nil)) (setq f (lambda () 1)) nil))` fa crescere la memoria residente di quasi 200 MB, circa 670 byte per giro; lo stesso ciclo con `(funcall (lambda () 1))` non la fa crescere.

Cosa succede. Una lambda tiene in vita l'ambiente in cui è nata. Se quell'ambiente contiene a sua volta la lambda, si forma un ciclo di contatori di riferimenti che non scende mai a zero. Succede quando una lambda viene assegnata a una variabile del `let` che la crea, quando una callback è salvata nel `let` che la definisce, o con un `defun` dentro una funzione. Non esiste un garbage collector che rompa questi cicli.

Effetto. Ogni ripetizione di un idioma comune perde memoria. In una sessione lunga, con hook e callback che creano chiusure, la memoria cresce senza limite.

Correzione proposta. Un raccoglitore di cicli, per esempio un mark-and-sweep periodico degli ambienti raggiungibili dall'ambiente globale e dalle pile delle fiber. Nel frattempo, documentare l'idioma da evitare.

INT-7 — `eq` è falso per una lambda, un atomo, una fiber o una primitiva confrontati con sé stessi

Gravità	MEDIA
Dove	`src/base/predicates.rs` (`primitive_eq_impl`, dietro `eq` ed `eql`, con rami solo per numeri, simboli, `nil`, forme, cons, vettori, mappe e stringhe)
Verifica	Riprodotto sulla revisione `f1ec558`: per una lambda e per un atomo `(eq x x)` dà `nil` ed `(equal x x)` dà `t`; per una fiber anche `equal` dà `nil` (INT-8).

Cosa succede. Per i tipi senza un ramo dedicato `eq` cade nel caso generico e restituisce `nil`, anche quando i due argomenti sono lo stesso oggetto. Non ha un ramo nemmeno la variante `ThreadHandle`, aggiunta in `f1ec558`, che oggi però nessun codice crea (INT-18).

Effetto. Viene violata la legge più elementare dell'identità. Il codice che cerca una funzione o un atomo per identità non la trova.

Correzione proposta. Un ramo `Arc::ptr_eq` per `Lambda`, `Atom`, `Fiber`, `Primitive` e `ThreadHandle`.

INT-8 — Una fiber non è uguale nemmeno a sé stessa

Gravità	MEDIA
Dove	`src/types.rs` (`impl PartialEq for SharedFiber`: `std::ptr::eq(self, other)` confronta gli indirizzi dei due **involucri**, non l'`Arc` condiviso)
Verifica	Riprodotto sulla revisione `f1ec558`: `(setq f (fiber (yield 1)))` e poi `(list (eq f f) (equal f f) (memq f (list f)))` dà `(nil nil nil)`.

Cosa succede. Ogni volta che una fiber viene letta da una variabile se ne clona l'involucro, cioè l'`Arc`. Due cloni hanno indirizzi diversi, quindi il confronto per indirizzo dell'involucro è sempre falso.
Effetto. Una fiber non si trova in nessuna lista: `memq`, `member`, `assoc`, `assq` e `remove-from-list` falliscono, quindi un registro di fiber non può toglierne una.
Correzione proposta. `Arc::ptr_eq(&self.0, &other.0)`.

INT-9 — `resume` di una fiber che sta già girando la uccide

Gravità	MEDIA
Dove	`src/base/fibers.rs` (`primitive_resume`), `src/types.rs` (`FiberState`, che ha solo `pending` e `is_done`)
Verifica	Riprodotto sulla revisione `f1ec558`: con `(setq f (fiber (resume f) (yield 1) 2))`, `(list (resume f) (fiber-done-p f) (resume f))` dà `(1 t nil)` invece di `(1 nil 2)`.

Cosa succede. Mentre una fiber gira, `resume` le toglie la pila dei frame, così un secondo `resume` non può eseguirla due volte. Ma il secondo `resume` trova la pila vuota e la marca come finita (`is_done = true`). Quando la fiber in corso fa `yield`, la pila viene salvata, però `is_done` resta vero e la fiber non riparte più. Il commento nel codice dice che togliere la continuazione impedisce a due `resume` di eseguirla due volte, e la documentazione di `FiberState::pending` che una pila vuota può voler dire che la fiber sta girando: nessuno dei due dice che il secondo `resume` la dichiara finita. Nella revisione `b2d000b`, quella che usa rsedit, il commento diceva che il secondo `resume` «non fa nulla».
Effetto. In rsedit, un comando che fa `resume` della fiber di un lavoro Lisp mentre il WorkManager la sta eseguendo la fa finire in silenzio: `LispJob::step` legge `fiber_is_done` e chiude il lavoro.
Correzione proposta. Uno stato esplicito «in esecuzione», distinto da «finita». Un `resume` su una fiber in esecuzione restituisce `nil` o un errore senza toccarla.

INT-10 — Un `yield` dentro `condition-case` non sospende la fiber: diventa un errore e il gestore lo intercetta

Gravità	MEDIA
Dove	`src/eval.rs` (`eval_special_form_or_call_step`, il ramo `"condition-case"`, che valuta la forma protetta con un `eval` senza il permesso di `yield`), `src/utils.rs` (`error_symbol`, `condition_matches`)
Verifica	Riprodotto sulla revisione `f1ec558`: `(fiber (condition-case e (yield 1) (error (setq got e) 'h)) 3)` restituisce 3 al primo `resume`, la fiber risulta finita e il gestore riceve `(error "YieldNotAllowed")`.

Cosa succede. Un `yield` è lecito solo dove il valutatore può registrare un punto di ripresa: un'istruzione del corpo o una forma di un `while`. Il corpo di `condition-case` non lo è, quindi il `yield` produce `YieldNotAllowed`. Il gestore generico `error` lo tratta come un errore qualsiasi.
Effetto. Una fiber che protegge con `condition-case` un passo che contiene un `yield` non si sospende mai. Il gestore d'errore parte e la fiber prosegue come se il passo fosse fallito. Il codice sembra corretto e si comporta in un altro modo.
Correzione proposta. Fare in modo che `condition_matches` non intercetti mai `YieldNotAllowed`, così l'errore arriva all'autore. Oppure aggiungere un tipo di frame per `condition-case`, così il `yield` diventa lecito anche lì. Il test `a_yield_inside_a_condition_case_is_refused` fissa il comportamento di oggi e va cambiato con la correzione.

INT-11 — Il backquote ignora la coda puntata, le mappe e l'annidamento

Gravità	MEDIA
Dove	`src/eval.rs` (`eval_backquote`, che restituisce le `Cons` e le mappe senza modifiche)
Verifica	Riprodotto sulla revisione `f1ec558`: con `b` uguale a 2, `` `(a . ,b) `` dà `(a . (unquote b))` invece di `(a . 2)`; `` `{k ,b} `` dà `{k (unquote b)}`; un backquote annidato dà `UndefinedFunction("unquote")`.

Cosa succede. Il lettore produce una `Cons` per una lista puntata e una mappa per `{…}`, e `eval_backquote` elabora solo le forme proprie e i vettori. Un livello di annidamento è documentato come limite, ma la coda puntata e le mappe no.
Effetto. Le macro che costruiscono liste puntate o mappe con il backquote producono strutture che contengono ancora `unquote`, e falliscono più tardi, lontano dalla causa.
Correzione proposta. Elaborare la coda di una `Cons` e i valori di una mappa nello stesso modo degli elementi di una lista; per l'annidamento, contare i livelli come in Emacs.

INT-12 — `format` interpreta le sequenze con backslash della stringa di formato

Gravità	MEDIA
Dove	`src/base/strings.rs` (`primitive_format`)
Verifica	Riprodotto sulla revisione `f1ec558`: `(format "\\b%s\\b" "w")` e `(format "C:\\Users\\%s" "me")` falliscono con «Wrong escape character». Che la sequenza `\n` scritta come due caratteri diventi un a capo viene dalla lettura del codice.

Cosa succede. Il lettore ha già trasformato le sequenze di escape della stringa letterale. `format` le interpreta una seconda volta e rifiuta quelle che non conosce.
Effetto. Non si può costruire con `format` un'espressione regolare con `\b` o `\s`, né un percorso Windows. Il testo che arriva da una variabile viene alterato.
Correzione proposta. Togliere l'interpretazione delle sequenze da `format`, che è compito del lettore.

INT-13 — `make-string` non ha un limite e non consuma carburante

Gravità	MEDIA
Dove	`src/base/strings.rs` (`primitive_make_string`)
Verifica	Riprodotto sulla revisione `f1ec558`: `(make-string (* 10000000 1000000) "a")` e un conteggio di 10^{23} danno l'errore `runtime-error` «make-string: unable to allocate requested string»; nell'analisi di rsedit il primo terminava il processo (uscita 134) e il secondo andava in panic («capacity overflow»). Che un conteggio concesso dall'allocatore riempia la memoria viene dalla lettura del codice.

Cosa succede. Da `90e0eb4` la lunghezza viene controllata contro l'overflow e la memoria è chiesta con `try_reserve_exact`, quindi una richiesta impossibile è un errore Lisp. Manca ancora un limite esplicito: una richiesta che l'allocatore concede, fino alla memoria fisica più lo swap, viene allocata e riempita un carattere alla volta, senza consumare carburante.
Effetto. `(make-string 2000000000 "a")` occupa 2 GB e due miliardi di giri che il carburante non ferma; con poca memoria libera il processo può essere ucciso dal sistema. Lo stesso schema si ritrova nelle primitive dell'editor (CMD-3, nel rapporto di rsedit).
Correzione proposta. Un limite esplicito (per esempio la dimensione massima di un buffer) con un errore `args-out-of-range`, e un consumo di carburante proporzionale alla lunghezza.

INT-14 — I numeri interi oltre 2^{63} vengono stampati saturati

Gravità	BASSA
Dove	`src/base/mod.rs` (`format_number`: `n as i64`, usata da `lisp_display`), `src/base/strings.rs` (`%d` di `primitive_format`)
Verifica	Riprodotto sulla revisione `f1ec558`: `(number-to-string 18446744073709551616)` dà `"9223372036854775807"`.

Cosa succede. Un numero intero viene stampato convertendolo a `i64`, conversione che satura. `%d` di `format` satura allo stesso modo.
Correzione proposta. Stampare come intero solo se `n.abs() < 2^53`, cioè quando la rappresentazione è esatta; altrimenti usare la forma decimale di `f64`.

INT-15 — `add-to-list` e funzioni affini: ambiente sbagliato, terzo argomento, costo quadratico

> **Gravità** BASSA
> **Dove** `src/base/lists.rs` (`primitive_add_to_list`, `primitive_append_to_list`, `primitive_remove_from_list`)
> **Verifica** Riprodotto sulla revisione `f1ec558`: dopo `(setq l '(a))` e `(defun h () (add-to-list 'l 'b))`, `(h)` lascia `l` uguale a `(a)`. Gli altri punti dalla lettura del codice.

Cosa succede. La lista viene letta con `get_variable`, che risale la catena degli ambienti, e riscritta con `set_variable` nell'ambiente del chiamante, anche quando esiste quella globale: è un difetto a sé, che correggere `setq` (INT-3) non risolve. Il terzo argomento non ha il significato di APPEND che ha in Emacs: viene aggiunto come elemento. Con più elementi, questi sono aggiunti in ordine inverso, senza controllare i duplicati tra gli argomenti stessi. Il costo è O(n·m) e non consuma carburante. Il messaggio d'errore di `remove-from-list` dice «add-to-list».
Correzione proposta. Aggiornare la variabile dove è definita (`update_variable`); allineare la firma a quella di Emacs, `(add-to-list SYMBOL ELEMENT &optional APPEND)`.

INT-16 — `memq` e `assq` confrontano con `equal`; `member` restituisce una copia

> **Gravità** BASSA
> **Dove** `src/base/lists.rs` (`primitive_memq`, `primitive_assq` e le loro docstring), `src/base/mod.rs` (`find_member`, `find_assoc`)
> **Verifica** Riprodotto sulla revisione `f1ec558`: `(memq "a" '("a"))` dà `("a")` mentre `(eq "a" "a")` dà `nil`. Il resto dalla lettura del codice.

Cosa succede. `memq` e `assq` usano lo stesso confronto strutturale di `member` e `assoc`, e la loro docstring dice ancora «eq is structural here», mentre `eq` ora confronta per identità. `member` e `memq` restituiscono una **copia** della coda (`proper_list(list[pos..].to_vec())`), non la coda condivisa: `(eq (member 2 l) (cdr l))` è falso e il costo è O(n).
Correzione proposta. Confronto con `eq` per `memq` e `assq`, docstring aggiornate, restituzione della coda condivisa scorrendo le `Cons`.

INT-17 — `string-to-number` accetta `inf` e `NaN`; `substring` oltre la fine restituisce la stringa vuota

> **Gravità** BASSA
> **Dove** `src/base/strings.rs` (`primitive_string_to_number`, `primitive_substring`)
> **Verifica** Riprodotto sulla revisione `f1ec558`: `string-to-number` dà `inf`, `NaN` e 100000 per `"inf"`, `"NaN"` e `"1e5"`; `(substring "abc" 5)` dà `""`.

Cosa succede. `string-to-number` usa `f64::from_str`, che accetta `"inf"`, `"NaN"` e `"1e5"`, mentre il lettore non legge nessuna delle tre come un numero (LET-4). `substring` con un inizio oltre la fine della stringa restituisce `""` invece di un errore `args-out-of-range`; un inizio maggiore della fine richiesta è un errore, ma `runtime-error`.
Correzione proposta. Un unico analizzatore di numeri, condiviso dal lettore e da `string-to-number`; un errore per gli indici fuori dai limiti.

INT-18 — `spawn` restituisce una forma vuota che è `eq` a `nil` ma non `equal`

> **Gravità** BASSA
> **Dove** `src/eval.rs` (`eval_special_form_or_call_step`, il ramo `"spawn"`)
> **Verifica** Riprodotto sulla revisione `f1ec558`:
> `(list (eq (spawn (lambda () 1)) nil) (equal (spawn (lambda () 1)) nil) (null (spawn (lambda () 1))))` dà `(t nil t)`.

Cosa succede. Il valore restituito è `LispExp::form(vec![])`, cioè una lista vuota, invece del `nil` canonico. `is_nil` la considera `nil`, il confronto strutturale no. Il commit `f1ec558`, «spawn returns a ThreadHandle value», aggiunge la variante `LispExp::ThreadHandle` ma non la usa: `spawn` restituisce ancora la forma vuota.

Correzione proposta. Restituire `LispExp::nil()`, oppure il `ThreadHandle` previsto, con i suoi rami in `eq` ed `equal` (INT-7).

INT-19 — Il controllo all'avvio lascia tre variabili globali

Gravità	BASSA
Dove	`src/handshake.rs` (`bootstrap_vm`)
Verifica	Riprodotto sulla revisione `f1ec558`: subito dopo `bootstrap_vm`, `(list verify-math fiber-step)` dà `(8 100)`.

Cosa succede. Il programma di verifica gira nell'ambiente globale e usa `setq`, e la documentazione di `bootstrap_vm` dice ora che le tre variabili restano legate. In caso di fallimento, l'errore restituito è `UncorrectFunctionDefinition`, che non dice nulla del problema.
Correzione proposta. Eseguirlo in un ambiente figlio e restituire un errore che descriva il controllo fallito.

INT-20 — `UncorrectFunctionDefinition` viene usato per errori che non c'entrano

Gravità	BASSA
Dove	`src/base/fibers.rs` (`primitive_resume`) e `src/base/atoms.rs` (`primitive_deref`, `primitive_reset`), per un lock avvelenato; `src/base/mod.rs` (`call_callable`), per un argomento che non è una funzione; `src/handshake.rs` (`bootstrap_vm`)
Verifica	Riprodotto sulla revisione `f1ec558`: `(funcall 5)` dà `(invalid-function "UncorrectFunctionDefinition")`; in rsedit lo stesso con `(with-current-buffer "*scratch*" (buffer-string))`.

Cosa succede. Lo stesso errore copre un lock avvelenato, un argomento che non è una funzione e un controllo d'avvio fallito. Da `039ca89` la sua documentazione elenca questi significati invece di separarli.
Correzione proposta. Usare `WrongArgumentType` per gli argomenti, in `call_callable` (in rsedit `with-current-buffer` può anche controllare da sé il suo argomento), e un errore dedicato per i lock avvelenati e per il controllo d'avvio.

INT-21 — Atomi senza aggiornamento atomico

Gravità	BASSA
Dove	`src/base/atoms.rs` (`make-atom`, `deref`, `reset`)
Verifica	Dalla lettura del codice.

Cosa succede. Esistono solo `deref` e `reset`, e ciascuno prende il lock per conto suo. Un incremento da due thread (`(reset a (+ 1 (deref a)))`) perde aggiornamenti, e nessuna primitiva permette di farlo in modo atomico. In più, gli esempi delle docstring di `deref` e `reset` creano l'atomo con `(atom 0)`, che è il predicato del tipo: il costruttore è `make-atom`.
Correzione proposta. Una primitiva `swap` che applica una funzione sotto il lock, oppure `compare-and-set`; negli esempi, `(make-atom 0)`.

INT-22 — Parametri chiamati `t` o `nil` sono accettati ma irraggiungibili

Gravità	BASSA
Dove	`src/utils.rs` (`parse_lambda_params`, `bind_lambda_args`), `src/eval.rs` (`eval_step_permitted`: i simboli che valutano a sé stessi sono risolti prima della ricerca)
Verifica	Riprodotto sulla revisione `f1ec558`: `((lambda (t) t) 5)` dà `t`.

Cosa succede. `(lambda (t) t)` viene definita senza errori, ma nel corpo `t` vale sempre `t`, perché i simboli che valutano a sé stessi non vengono cercati nell'ambiente. Lo stesso vale per `nil` e per le keyword. Accettano questi nomi anche `let`, `let*` (`parse_let_binding`) e `setq`.
Correzione proposta. Rifiutare questi nomi in `parse_lambda_params`, in `parse_let_binding` e in `setq`.

INT-23 — `function-doc` contiene un `unreachable!`

Gravità BASSA
Dove `src/base/functions.rs` (`primitive_function_doc`)
Verifica Riprodotto sulla revisione `f1ec558` con un host che lega un simbolo nello spazio delle funzioni: dopo `env.set_function("x", LispExp::symbol("car"))`, `(function-doc 'x)` va in panic.

Cosa succede. `primitive_function_doc` gestisce solo lambda e primitive e finisce con `_ => unreachable!()`. Dentro risp e in rsedit nessuno inserisce altro nello spazio delle funzioni; ma risp è ora un crate a sé, `Env::set_function` è pubblica e accetta qualunque `LispExp`, e `call_callable` segue già un simbolo legato lì come un alias. Un host che lo faccia manda `function-doc` in panic.
Correzione proposta. `_ => Ok(LispExp::nil())`.

INT-24 — `if` valuta solo la prima forma del ramo else e ignora le altre in silenzio

Gravità MEDIA
Dove `src/eval.rs` (`eval_special_form_or_call_step`, il ramo `"if"`)
Verifica Riprodotto sulla revisione `f1ec558`: `(setq x 0) (if nil 1 (setq x 1) (setq x 2)) x` dà 1; in Emacs dà 2.

Cosa succede. Il ramo `"if"` controlla che ci siano la condizione e il ramo then, valuta la condizione e poi continua, come chiamata in coda, con il ramo then oppure con `args[2]`. Gli argomenti da `args[3]` in poi non vengono mai valutati, e nessun errore lo dice. In Emacs Lisp il ramo else è un `progn` implicito: tutte le sue forme vengono valutate, e `if` restituisce il valore dell'ultima.
Come riprodurlo.

```
(setq x 0)
(if nil 1 (setq x 1) (setq x 2))   ; 1, e x vale 1; in Emacs 2 e 2
```

Effetto. Codice scritto come in Emacs perde in silenzio una parte del ramo else. È successo in rsedit: in `vim--pending-key`, nel modulo `vim`, un `if` aveva `(vim--reset)` come seconda forma del ramo else, e un conteggio davanti a `gg` o `ge` restava al comando dopo (`4gg` seguito da `dd` cancellava quattro righe). Il modulo è stato corretto in rsedit con `ef75b23`; l'interprete no.
Correzione proposta. Valutare le forme del ramo else come quelle di `progn`, con `eval_body_step`, che dà anche il trattamento di `yield`; in alternativa rifiutare un `if` con più di tre argomenti con un errore, che almeno non fa sparire forme in silenzio. Un test per ciascun caso: il valore restituito e gli effetti di tutte le forme del ramo else.

4 Documentazione e test

DOC-1 — Il manuale dell'interprete descrive come attuali cinque difetti corretti

Gravità BASSA
Dove `doc/architettura-interprete.typ`
Verifica Dalla lettura del documento sulla revisione `f1ec558`.

Cosa succede. Il manuale è entrato in `90e0eb4` e non è cambiato da allora. Descrive ancora il lettore ricorsivo, con `parse_list` e `UnexpectedDot` e senza il limite `TooDeep` (LET-1, LET-2, LET-3), il `ctx.begin_unwind()` di `unwind-protect` (INT-1) e `dotimes` e `dolist` senza carburante (INT-5), con i rimandi a quelle schede. Dice che i percorsi sono relativi a `core/src/lisp/`, mentre in risp sono relativi alla radice del repository. E la sua lista di controllo dice di togliere la scheda di un difetto corretto, mentre questo rapporto la segna come risolta.
Correzione proposta. Riscrivere le sezioni del lettore, di `unwind-protect` e dei cicli sul codice di oggi, togliendo i rimandi alle schede risolte; correggere la radice dei percorsi e la lista di controllo.

DOC-2 — Un file di test non viene mai compilato

Gravità BASSA
Dove `src/tests/reader_tests.rs`, `src/tests/mod.rs`
Verifica Riprodotto sulla revisione `f1ec558`: dichiarato in `src/tests/mod.rs`, il file non compila, con quattro errori su `ParserError::UnexpectedDot`.

Cosa succede. Il file è arrivato da rsedit insieme all'interprete e contiene i test delle correzioni proposte nelle schede del lettore. Nessun `mod` lo dichiara, quindi `cargo test` non lo compila. Dichiarato, non compilerebbe: usa `ParserError::UnexpectedDot`, che `3b7cbac` ha sostituito con `UnexpectedToken(".")`. E alcune sue asserzioni descrivono come corretto il comportamento di difetti ancora aperti (LET-4, LET-5, LET-6).
Correzione proposta. Dichiararlo, sostituire `UnexpectedDot`, e per i difetti ancora aperti segnare i test con `#[ignore]` e un `TODO(ID)`, oppure toglierli finché la scheda non si chiude.

DOC-3 — `cargo clippy --all-targets` si ferma su un letterale dei test

Gravità BASSA
Dove `src/tests/parser_tests.rs` (`test_edge_negative_numbers`)
Verifica Riprodotto sulla revisione `f1ec558`: `cargo clippy --all-targets` termina con «approximate value of `f{32, 64}::consts::PI` found», una lint di livello deny.

Cosa succede. Il letterale `-3.14` in un test del lettore viene scambiato da clippy per π. Il difetto viene dalla scheda DOC-4 del rapporto di rsedit, risolta per rsedit: il test è arrivato qui con l'interprete.
Correzione proposta. Usare un altro letterale, per esempio `-2.5`.

5 Appendici

5.1 Le schede risolte

Le schede risolte dopo l'analisi, con il commit che le ha risolte. Il testo completo di ciascuna sta nel rapporto di rsedit, `doc/rapporto-problemi.typ`, fino alla revisione `3d846f1`; che cosa è cambiato, e perché, nel messaggio del commit.

ID	Problema	Gravità	Risolto in
LET-1	Un input che finisce dentro un simbolo, con una lista aperta, esaurisce la memoria	ALTA	`3b7cbac`
LET-2	Un token che non può cominciare un'espressione manda il lettore in panic	ALTA	`90e0eb4`
LET-3	Un annidamento profondo esaurisce lo stack del lettore	MEDIA	`3b7cbac`
INT-1	Un ciclo che contiene `unwind-protect` non esaurisce mai il carburante	ALTA	`b2d000b`
INT-5	`dotimes` con il corpo vuoto non consuma carburante	MEDIA	`90e0eb4`

5.2 Differenze da Emacs che non sono difetti

Alcuni comportamenti differiscono da Emacs per scelta, o comunque senza produrre risultati sbagliati. Sono elencati qui perché chi conosce Emacs non li scambi per errori.

- `%` ha la stessa semantica di `mod`: il segno del risultato segue il divisore. In Emacs `%` è il resto e segue il dividendo: `(% -7 2)` dà 1 qui e −1 in Emacs.
- `out-of-fuel` non si intercetta con `condition-case`. È voluto: un ciclo infinito non deve potersi proteggere dal meccanismo che lo ferma. I gestori di `unwind-protect` girano comunque.
- Il lettore rifiuta un'espressione annidata più di 256 livelli, con l'errore `TooDeep`: è il limite che ha risolto LET-3. Emacs legge strutture più profonde.

5.3 Parti esaminate e trovate solide

Nell'analisi tutte le forme speciali dell'interprete sono state esercitate con sintassi malformata (181 322 forme valutate dal fuzzer) senza trovare un panic.